ES-CS301 — Data Structure & Algorithms

Semester III • 3L:0T:4P • 3 Credits

Module 3 Exam Notes: Linked Lists and Tree Data Structures

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1 Linked Lists Fundamentals and Singly Linked List (SLL) . . . 2
1.1 Limitations of Arrays vs Advantages of Linked Lists . . . 2
1.2 Singly Linked List Algorithmic Operations . . . 2
2 Linked Representation of Stack and Queue & Header Nodes . . . 3
2.1 Linked Stack and Linked Queue . . . 3
2.2 Header Nodes in Linked Lists . . . 3
3 Doubly Linked Lists (DLL) and Circular Linked Lists (CLL) . . . 4
3.1 Doubly Linked List (DLL) . . . 4
3.2 Circular Linked Lists (CLL) . . . 4
3.3 Polynomial Representation and Addition using Linked Lists . . . 4
4 Tree Terminologies and Binary Tree Foundations . . . 5
4.1 Mathematical Properties of Binary Trees . . . 5
4.2 Binary Tree Classifications . . . 6
4.3 Binary Tree Traversals . . . 6
5 Threaded Binary Tree . . . 6
6 Binary Search Tree (BST) . . . 7
6.1 BST Operations and Recursive Algorithms . . . 7
7 AVL Tree (Self-Balancing Binary Search Tree) . . . 8
7.1 AVL Rebalancing Rotations . . . 8
8 Applications of Binary Trees: Expression Trees and Huffman Coding . . . 9
8.1 Expression Trees . . . 9
8.2 Huffman Coding (Greedy Data Compression) . . . 9
9 Multi-Way Search Trees: B-Tree and B+ Tree . . . 10
9.1 B-Tree of Order m . . . 10
9.2 B+ Tree and Comparison with B-Tree . . . 10

1. Linked Lists Fundamentals and Singly Linked List (SLL)

Definition: Singly Linked List (SLL) Architecture

A **Singly Linked List** is a dynamic linear data structure composed of distinct nodes allocated at arbitrary non-contiguous physical memory locations. Each node consists of two essential components:

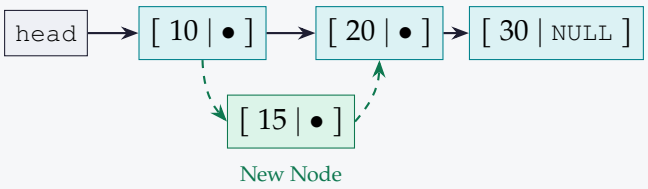
1. **Data Field:** Stores the actual information payload of type T.
2. **Link (Next Pointer):** Stores the explicit physical memory address of the logically succeeding node.

The list is anchored by an external pointer `head` referencing the first node. The terminal node's link contains `NULL`.

1.1 Limitations of Arrays vs Advantages of Linked Lists

- **Memory Allocation:** Arrays require fixed, compile-time contiguous memory allocation. Linked lists allocate memory dynamically on the heap at runtime via `malloc() / free()`, growing and shrinking on demand.
- **Insertion and Deletion Overhead:** Inserting or deleting an element at index 0 in an array of size n requires shifting all n subsequent elements ($\mathcal{O}(n)$ time). In a linked list, inserting or deleting at the head requires updating pointer references, which executes in strictly $\mathcal{O}(\mathbf{1})$ time without moving physical payloads.
- **Memory Fragmentation:** Arrays can suffer allocation failure due to external memory fragmentation even when total free RAM exceeds the array size; linked lists utilize scattered memory chunks effectively.

Memory Representation and Pointer Rewiring in a Singly Linked List



1.2 Singly Linked List Algorithmic Operations

Algorithms for Insertion and Deletion in SLL

1. Insertion at Beginning ($\mathcal{O}(1)$):

```
newNode = allocateNode(val);
newNode->next = head;
head = newNode;
```

2. Insertion at End ($\mathcal{O}(n)$ without tail pointer, $\mathcal{O}(1)$ with tail):

```
newNode = allocateNode(val); newNode->next = NULL;
if (head == NULL) { head = newNode; return; }
```


© Biraj

```
ptr = head;
while (ptr->next != NULL) ptr = ptr->next;
ptr->next = newNode;
```

3. Deletion at Beginning ($\mathcal{O}(1)$):

```
if (head == NULL) report "Underflow";
temp = head;
head = head->next;
free(temp);
```

4. Deletion of a Node with Value X ($\mathcal{O}(n)$):

```
if (head == NULL) return;
if (head->data == X) { temp = head; head = head->next; free(temp); return; }
curr = head; prev = NULL;
while (curr != NULL && curr->data != X) { prev = curr; curr = curr->next; }
if (curr == NULL) report "Key Not Found";
else { prev->next = curr->next; free(curr); }
```

2. Linked Representation of Stack and Queue & Header Nodes

2.1 Linked Stack and Linked Queue

- **Linked Stack:** Implemented by inserting and deleting strictly at the head node of the linked list:
  - Push(x) $\equiv$ Insert at Beginning $\implies \mathcal{O}(\mathbf{1})$
  - Pop() $\equiv$ Delete from Beginning $\implies \mathcal{O}(\mathbf{1})$
  - Overflow never occurs unless the system runs out of heap memory.
- **Linked Queue:** Maintained via two external pointers: `front` and `rear`:
  - Enqueue(x) $\equiv$ Insert at End (via `rear->next` = newNode; `rear` = newNode) $\implies \mathcal{O}(\mathbf{1})$
  - Dequeue() $\equiv$ Delete at Head (via `front` = `front->next`) $\implies \mathcal{O}(\mathbf{1})$

2.2 Header Nodes in Linked Lists

A **Header Node** is a dummy node placed at the very beginning of the linked list before the first actual data node:

- **Grounded Header List:** Header node's link points to the first data node; the last node's link contains `NULL`.
- **Circular Header List:** The last node's link points back to the header node itself.
- **Key Advantages:**
  1. Eliminates special-case coding for operations at the head (head pointer never needs re-assignment).
  2. The list is never truly empty (`head != NULL` always holds; an empty list has `head->next == NULL` or `head->next == head`).

© Biraj

3. The header node's data field can store metadata (e.g., total node count, maximum element).

3. Doubly Linked Lists (DLL) and Circular Linked Lists (CLL)

3.1 Doubly Linked List (DLL)

In a **Doubly Linked List**, each node contains three fields: `prev` pointer, `data`, and `next` pointer.

- **Invariant:** `p->next->prev` $==$ `p` and `p->prev->next` $==$ `p`.
- **Bidirectional Traversal:** Allows forward and reverse scanning from any arbitrary node.
- $\mathcal{O}(1)$ **Deletion Given Node Pointer:** Deleting node P does not require searching for its predecessor:

```
P->prev->next = P->next;
if (P->next != NULL) P->next->prev = P->prev;
free(P);
```

- **Trade-off:** Extra memory overhead for backward pointers ($2\times$ pointer overhead per node) and additional pointer adjustments during insertions.

3.2 Circular Linked Lists (CLL)

In a **Circular Singly Linked List**, the last node's link points back to the first node.

- **Tail Pointer Paradigm:** By maintaining a single pointer `tail` to the last node:
  - Last node is accessible in $\mathcal{O}(1)$ (`tail`).
  - First node is accessible in $\mathcal{O}(1)$ (`tail->next`).
  - Both insertion at head and insertion at tail execute in strictly $\mathcal{O}(\mathbf{1})$ time!
- **Applications:** Continuous cycling processes, round-robin CPU scheduling, multiplayer turn games.

3.3 Polynomial Representation and Addition using Linked Lists

A single-variable polynomial $P(x) = a_n x^{e_n} + a_{n-1}x^{e_{n-1}} + \cdots + a_0 x^{e_0}$ is represented as a linked list where each node contains: `coeff` (coefficient), `exp` (exponent), and `next` pointer, maintained in strictly decreasing order of exponents.

Algorithm: Polynomial Addition using Singly Linked Lists

Algorithm AddPolynomials(P_1, P_2)**:**
1. Initialize pointers $p = P_1$, $q = P_2$, and an empty resultant list P_3.
2. While $p \neq$ NULL and $q \neq$ NULL:
   - **Case 1** ($p \to$ **exp** $==$ $q \to$ **exp**)**:** Compute sum $= p \to$ coeff $+ q \to$ coeff. If sum $\neq 0$, insert term $(\text{sum}, p \to \text{exp})$ into P_3. Advance $p = p \to$ next, $q = q \to$ next.

© Biraj

- **Case 2 ($p \to$ exp $> q \to$ exp):** Insert term $(p \to \text{coeff}, p \to \text{exp})$ into P_3. Advance $p = p \to \text{next}$.
- **Case 3 ($p \to$ exp $< q \to$ exp):** Insert term $(q \to \text{coeff}, q \to \text{exp})$ into P_3. Advance $q = q \to \text{next}$.

3. Copy any remaining terms from p or q to the tail of P_3. Return P_3.

Time Complexity: $\mathcal{O}(\mathbf{m} + \mathbf{n})$ where m, n are the number of non-zero terms in P_1, P_2.

4. Tree Terminologies and Binary Tree Foundations

Definition: Tree Data Structure and Core Terminologies

A **Tree** is a non-linear, hierarchical data structure consisting of a collection of nodes connected by directed or undirected edges, such that there exists a unique root node R and no cycles.

- **Root:** The topmost ancestor node with in-degree 0 (no incoming parent edge).
- **Leaf (Terminal Node):** A node with out-degree 0 (no child nodes).
- **Degree of a Node:** The total number of subtrees or immediate children attached to that node.
- **Degree of a Tree:** The maximum degree among all nodes in the tree.
- **Level and Depth:** The root is at Level 0 (or 1). The depth of a node is the length of the path from the root to that node.
- **Height of a Tree (h):** The maximum depth among all nodes in the tree (length of the longest path from root to leaf).

4.1 Mathematical Properties of Binary Trees

A **Binary Tree** is a finite tree in which every node has at most two children, distinguished as **left child** and **right child**.

Core Mathematical Theorems on Binary Trees

1. Maximum number of nodes at level i (with root at level 0):

$$N_{\max}(i) = 2^i \tag{1}$$

2. Maximum total nodes in a binary tree of height h:

$$N_{\max}(h) = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1 \tag{2}$$

3. Minimum height of a binary tree with n nodes:

$$h_{\min} = \lceil \log_2(n+1) \rceil - 1 \tag{3}$$

© Biraj

4. **Leaf vs Degree-2 Nodes Theorem:** In any non-empty binary tree, the number of leaf nodes n_0 is exactly one more than the number of nodes with degree two (n_2):

$$\mathbf{n_0 = n_2 + 1} \tag{4}$$

Proof: Let total nodes be $n = n_0 + n_1 + n_2$. Total edges in a tree of n nodes is $E = n - 1$. Also, each degree-1 node contributes 1 outgoing branch and each degree-2 node contributes 2 outgoing branches: $E = n_1 + 2n_2$. Equating the two expressions:

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \implies n_0 - 1 = n_2 \implies \mathbf{n_0 = n_2 + 1}$$

4.2 Binary Tree Classifications

- **Full (Strict / Proper) Binary Tree:** Every internal node has exactly two children (0 or 2 children; no node has degree 1).
- **Complete Binary Tree:** All levels are completely filled except possibly the last level, where nodes are filled strictly from left to right without gaps. Foundation of array-based Binary Heaps.
- **Perfect Binary Tree:** All internal nodes have two children and all leaves reside at the same maximum depth (contains exactly $2^{h+1} - 1$ nodes).
- **Degenerate (Skewed) Binary Tree:** Every internal node has only one child, degrading to a linear linked list ($\mathcal{O}(n)$ height).

4.3 Binary Tree Traversals

1. **Preorder Traversal (D-L-R):** Process Data → Traverse Left subtree → Traverse Right subtree.
2. **Inorder Traversal (L-D-R):** Traverse Left subtree → Process Data → Traverse Right subtree. Produces sorted sequence for a BST!
3. **Postorder Traversal (L-R-D):** Traverse Left subtree → Traverse Right subtree → Process Data. Used for deleting tree bottom-up and expression evaluation.
4. **Level-Order Traversal (BFS):** Visit nodes level by level from top to bottom, left to right, implemented using a FIFO Queue.

5. Threaded Binary Tree

Definition: Threaded Binary Tree

In an n-node binary tree, exactly $2n$ pointer fields exist, of which $n-1$ point to children and $n+1$ are `NULL`. A **Threaded Binary Tree** utilizes these wasted null pointers:

- A `NULL` left pointer is converted into a **thread** pointing to the node's **Inorder Predecessor**.
- A `NULL` right pointer is converted into a **thread** pointing to the node's **Inorder Successor**.

© Biraj

Boolean tags (`lTag`, `rTag`) indicate whether the pointer field is a normal child link (0) or a thread (1).

Primary Advantage: Enables complete Inorder traversal without requiring recursive calls or an auxiliary stack, utilizing $\mathcal{O}(\mathbf{1})$ auxiliary memory space.

6. Binary Search Tree (BST)

Definition: Binary Search Tree (BST) Invariant

A **Binary Search Tree (BST)** is a binary tree in which every node x satisfies the BST ordering property:
- For all nodes y in the left subtree of x: $\text{Key}(y) < \text{Key}(x)$.
- For all nodes z in the right subtree of x: $\text{Key}(z) > \text{Key}(x)$.

Duplicate keys are either disallowed or handled uniformly in right subtrees.

6.1 BST Operations and Recursive Algorithms

Core Binary Search Tree Algorithms

1. Algorithm BST_Search($root, K$):

```
if (root == NULL || root->data == K) return root;
if (K < root->data) return BST_Search(root->left, K);
return BST_Search(root->right, K);
```

2. Algorithm BST_Insert($root, K$):

```
if (root == NULL) return createNode(K);
if (K < root->data) root->left = BST_Insert(root->left, K);
else if (K > root->data) root->right = BST_Insert(root->right, K);
return root;
```

3. Algorithm BST_Delete($root, K$):

```
if (root == NULL) return root;
if (K < root->data) root->left = BST_Delete(root->left, K);
else if (K > root->data) root->right = BST_Delete(root->right, K);
else {
    // Case 1: Leaf node or Case 2: One child
    if (root->left == NULL) {
        temp = root->right; free(root); return temp;
    } else if (root->right == NULL) {
        temp = root->left; free(root); return temp;
    }
    // Case 3: Two children
    temp = findMin(root->right); // Inorder Successor
    root->data = temp->data;
```


© Biraj

```
    root->right = BST_Delete(root->right, temp->data);
}
return root;
```

Complexity: Average $\mathcal{O}(\log_2 \mathbf{n})$, Worst (skewed tree) $\mathcal{O}(\mathbf{n})$.

7. AVL Tree (Self-Balancing Binary Search Tree)

Definition: AVL Tree and Balance Factor

An **AVL Tree** is a strictly height-balanced Binary Search Tree where the **Balance Factor** (BF) of every node v is constrained to $\{-1, 0, +1\}$:

$$BF(v) = h(\text{left subtree}) - h(\text{right subtree}) \in \{-1, 0, +1\} \tag{5}$$

If an insertion or deletion causes $|BF(v)| \geq 2$, the tree is rebalanced locally via rotations.

7.1 AVL Rebalancing Rotations

1. **LL Rotation (Single Right Rotation):** Applied when insertion occurs in the left subtree of the left child ($BF = +2$, child $BF = +1$).
2. **RR Rotation (Single Left Rotation):** Applied when insertion occurs in the right subtree of the right child ($BF = -2$, child $BF = -1$).
3. **LR Rotation (Double Rotation):** Insertion in the right subtree of the left child ($BF = +2$, child $BF = -1$). First Left-rotate left child, then Right-rotate parent.
4. **RL Rotation (Double Rotation):** Insertion in the left subtree of the right child ($BF = -2$, child $BF = +1$). First Right-rotate right child, then Left-rotate parent.

Step-by-Step AVL Construction Walkthrough

Construct an AVL tree by inserting keys: $10, 20, 30, 40, 50, 25$.

- Insert $10, 20$: Tree balanced.
- Insert 30: Path $10 \to 20 \to 30$ causes $BF(10) = -2$ (RR violation). Perform **RR (Left Rotation)** on 10: 20 becomes root, with children 10 (left) and 30 (right).
- Insert 40: Attached as right child of 30. Balanced ($BF(20) = -1$).
- Insert 50: Path $30 \to 40 \to 50$ causes $BF(30) = -2$ (RR violation). Perform **RR (Left Rotation)** on 30: 40 takes 30's position, children $30, 50$.
- Insert 25: Attached as left child of 30. Root 20 now has $BF(20) = -2$, child 40 has $BF(40) = +1$ (RL violation). Perform **RL Double Rotation**: Right-rotate 40 then Left-rotate 20. New root is 30, left subtree rooted at 20 (children $10, 25$), right subtree rooted at 40 (child 50). Tree is perfectly balanced!

© Biraj

Height Bound and Complexity of AVL Trees

The minimum number of nodes $N(h)$ in an AVL tree of height h satisfies the Fibonacci recurrence:

$$N(h) = N(h-1) + N(h-2) + 1, \quad N(0) = 1,\ N(1) = 2 \tag{6}$$

Solving this relation yields an asymptotic upper bound on height:

$$h < 1.44 \log_2(n+2) \implies \mathbf{h} = \mathcal{O}(\log_2 \mathbf{n}) \tag{7}$$

Consequently, Search, Insert, and Delete operations in an AVL tree are **guaranteed strictly** $\mathcal{O}(\log \mathbf{n})$ in both average and worst cases!

8. Applications of Binary Trees: Expression Trees and Huffman Coding

8.1 Expression Trees

An **Expression Tree** is a binary tree representing an arithmetic or logical formula:

- **Leaves:** Distinct operands (variables or numerical constants).
- **Internal Nodes:** Binary operators $(+, -, *, /, \wedge)$.
- **Traversals:** Inorder traversal with parentheses yields the equivalent **Infix** formula; Preorder traversal yields **Prefix**; Postorder traversal yields **Postfix**.
- **Construction Algorithm from Postfix:** Scan postfix expression token by token using a stack of tree node pointers. For an operand, create a leaf node and push pointer. For operator $\odot$, create an operator node, pop two nodes as right and left children, and push the newly formed subtree.

8.2 Huffman Coding (Greedy Data Compression)

Huffman Algorithm for Optimal Prefix Codes

Given an alphabet of n symbols with given occurrence frequencies:

1. Create a Min-Priority Queue containing n leaf nodes, each initialized with a symbol and its frequency.
2. While the priority queue contains more than one node:
   - $left$ = Extract-Min()
   - $right$ = Extract-Min()
   - Allocate a new internal node Z with freq(Z) = freq$(left)$ + freq$(right)$.
   - Set $Z \to$ left = $left$ and $Z \to$ right = $right$.
   - Insert(Z) back into the Min-Priority Queue.
3. The remaining single node is the root of the **Huffman Tree**.
4. Assign binary bit `'0'` to each left branch and `'1'` to each right branch. Paths from root to leaves produce unambiguous **Prefix Codes** (no code is a prefix of another).

Time Complexity: $\mathcal{O}(\mathbf{n} \log_2 \mathbf{n})$ using a binary min-heap.

© Biraj

9. Multi-Way Search Trees: B-Tree and B+ Tree

9.1 B-Tree of Order m

Designed for secondary storage (disk systems) to minimize slow disk I/O operations by maximizing node branching factor.

Axiomatic Properties of B-Tree of Order m

1. Every node has at most m children and $m-1$ keys.
2. The root node has at least 2 children (unless it is a leaf).
3. Every internal node (except root) has at least $\lceil m/2 \rceil$ children and $\lceil m/2 \rceil - 1$ keys.
4. All leaf nodes reside at the exact same depth/level.
5. A non-leaf node with k children contains $k-1$ keys acting as separation boundaries.

B-Tree Algorithmic Operations: Insertion and Deletion

1. B-Tree Insertion Algorithm:

- Search to find the appropriate leaf node for key K.
- If leaf contains $< m-1$ keys, insert K into sorted position.
- **Node Splitting (Overflow):** If leaf already contains $m-1$ keys, inserting K causes overflow (m keys). Split node around the **median key** at position $\lceil m/2 \rceil$:
  - Keys left of median form a new left leaf.
  - Keys right of median form a new right leaf.
  - Median key is **promoted** to the parent node.
  - If parent overflows, propagate splitting upward recursively (if root splits, tree height increases by 1).

2. B-Tree Deletion Algorithm:

- If key is in an internal node, replace it with its inorder predecessor or successor, reducing problem to deletion from a leaf.
- If leaf has $\geq \lceil m/2 \rceil$ keys, delete directly.
- **Underflow Handling ($< \lceil m/2 \rceil - 1$ keys):**
  - **Borrowing (Rotation):** If immediate sibling has $> \lceil m/2 \rceil - 1$ keys, borrow a key via rotation through parent.
  - **Merging:** If both siblings have minimum keys, merge the node with a sibling and the intervening parent separator key (may cause underflow in parent).

9.2 B+ Tree and Comparison with B-Tree

- **B+ Tree Structure:** In a B+ tree, all data records and actual keys are stored **strictly in leaf nodes**. Internal nodes store only search routing keys.
- **Linked Leaves:** All leaf nodes are linked sequentially via pointers to form a doubly linked list, enabling extremely fast $\mathcal{O}(k)$ range queries.
- **Database Indexing Standard:** B+ Trees are universally used in relational database engines (MySQL, PostgreSQL, Oracle) and file systems (NTFS, ext4).

© Biraj

Quick Revision — Module 3

Linked Lists and Tree Data Structures | ES-CS301

Key Formulas, Tree Bounds and Complexity at a Glance

Binary Tree Max Nodes Level i / Total	$N_{\max}(i) = 2^i, \quad N_{\text{total}}(h) = 2^{h+1} - 1$
Leaf Nodes vs Degree-2 Nodes	$n_0 = n_2 + 1$ (Universal Binary Tree Invariant)
Binary Tree Minimum Height	$h_{\min} = \lceil \log_2(n+1) \rceil - 1 \implies \Omega(\log n)$
AVL Balance Factor	$BF(v) = h_L(v) - h_R(v) \in \{-1, 0, +1\}$
AVL Maximum Height Bound	$h < 1.44 \log_2 n \implies$ Guaranteed $\mathcal{O}(\log n)$
SLL Head Operations vs Search	Insert/Delete Head: $\mathcal{O}(1)$, Search/Tail: $\mathcal{O}(n)$
BST Search / Insert / Delete	Average: $\mathcal{O}(\log n)$, Worst (skewed): $\mathcal{O}(n)$
B-Tree Key Bounds per Node	$\lceil m/2 \rceil - 1 \leq$ keys $\leq m - 1$, Search: $\mathcal{O}(\log_m n)$

One-Line Definitions

- **Singly Linked List:** A dynamic linear data structure composed of nodes storing data and a single pointer to the successor.
- **Doubly Linked List:** A linked list where each node contains pointers to both its forward and backward adjacent nodes.
- **Circular Linked List:** A linked list whose terminal node points back to the first node instead of storing `NULL`.
- **Header Node:** A dummy lead node placed before actual data elements to simplify edge cases and eliminate head pointer updates.
- **Complete Binary Tree:** A binary tree completely filled at all levels except possibly the last, which is filled left-to-right.
- **Threaded Binary Tree:** A binary tree that replaces null leaf pointers with threads to inorder predecessors and successors.
- **Binary Search Tree (BST):** A binary tree where all left descendants are smaller and all right descendants are greater than the node.
- **AVL Tree:** A height-balanced BST where the heights of left and right subtrees of every node differ by at most 1 ($|BF| \leq 1$).

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. State and prove the relationship between leaf nodes and degree-2 nodes in any non-empty binary tree.

In any binary tree, $n_0 = n_2 + 1$. Proof: Let total nodes be $n = n_0 + n_1 + n_2$. Total edges in a tree of n nodes is $E = n - 1$. Also, each degree-1 node contributes 1 outgoing branch and each degree-2 node contributes 2: $E = n_1 + 2n_2$. Setting $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$ simplifies directly to $n_0 = n_2 + 1$.

Q2. Why does Inorder traversal of a Binary Search Tree produce sorted output?

Inorder traversal follows the Left-Data-Right (L-D-R) sequence. Because the BST invariant guarantees that all left subtree nodes are smaller than the root, and the root is smaller than all right subtree nodes, recursively visiting Left, then Root, then Right naturally produces a monotonically ascending sequence.

Q3. Explain the four rebalancing rotations of an AVL tree.

Single Right (LL) fixes left-heavy imbalance caused by insertion in left subtree of left child. Single Left (RR) fixes right-heavy imbalance. Left-Right (LR) double rotation fixes left-heavy node with right-heavy child via left rotation on child followed by right on parent. Right-Left (RL) performs right rotation on child followed by left on parent.

Q4. How does a B+ Tree differ fundamentally from a B-Tree?

In a B-tree, keys and associated data records are stored in both internal and leaf nodes. In a B+ tree, all data records are stored strictly in leaf nodes; internal nodes store only routing index keys. Furthermore, leaf nodes in a B+ tree are linked sequentially into a linked list for rapid range scanning.

Q5. What are the advantages of maintaining a Tail pointer in a Circular Linked List?

With only a tail pointer, the last node is directly accessed in $\mathcal{O}(1)$ via `tail`, and the first node is accessed in $\mathcal{O}(1)$ via `tail->next`. This allows insertion at the beginning and insertion at the end to both execute in strictly $\mathcal{O}(1)$ time without traversing the list.

Q6. What is the primary motivation for using Threaded Binary Trees?

In a binary tree of n nodes, $n + 1$ pointer fields are wasted as `NULL`. Threaded trees replace these null pointers with threads pointing to inorder predecessors and successors, permitting full inorder traversal without recursion or an auxiliary stack ($\mathcal{O}(1)$ extra space).

Q7. What is the worst-case time complexity of searching in a BST, and when does it occur?

The worst-case search complexity is $\mathcal{O}(n)$. This occurs when keys are inserted in strictly sorted or reverse-sorted order, causing the BST to degenerate into a skewed linear chain where height $h = n - 1$.

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. Why are B-Trees preferred over Binary Search Trees for database indexes on disk?

Disk access is significantly slower than RAM access. A BST has high height ($\mathcal{O}(\log_2 n)$), requiring many sequential disk seeks. A B-Tree has a high branching factor m (order $m \approx 100$ to 1000), keeping height very low ($\mathcal{O}(\log_m n)$), so only 2 to 3 disk I/O reads are needed to retrieve any record.

Q9. Explain the three cases of node deletion in a Binary Search Tree.

Case 1: Leaf node (simply remove and nullify parent's pointer). Case 2: Node has one child (bypass node by connecting parent directly to child). Case 3: Node has two children (replace value with inorder successor or predecessor, then delete that node which has at most one child).

Q10. What is the purpose of a Header Node in a linked list?

A header node simplifies boundary conditions: it ensures the list is never empty (head pointer never changes) and eliminates special-case logic for inserting or deleting at the front of the list.

Summary Matrix: Comprehensive Comparison of Tree Data Structures

Tree Structure	Structural Invariant	Search Complexity	Primary Application Domain
Binary Tree	**At most 2 children per node**	$\mathcal{O}(n)$	Expression trees, hierarchical parsing, Huffman coding.
Threaded Binary Tree	**Null pointers store inorder links**	**Traversal:** $\mathcal{O}(1)$ **space**	Stackless tree traversal in memory-constrained systems.
Binary Search Tree (BST)	**Left** $<$ **Root** $<$ **Right**	**Avg:** $\mathcal{O}(\log n)$, **Worst:** $\mathcal{O}(n)$	Dynamic searching, symbol tables, dictionary ADT.
AVL Tree	**Strict balance:** $\lvert BF\rvert \leq 1$	**Guaranteed** $\mathcal{O}(\log n)$	Search-intensive in-memory datasets and lookup tables.
B-Tree	**Multi-way:** $\lceil m/2 \rceil \leq$ **children** $\leq m$	$\mathcal{O}(\log_m n)$	File systems, relational databases, storage indexing.
B+ Tree	**Data in leaves; leaves linked**	$\mathcal{O}(\log_m n)$	Database query engines, fast sequential range retrieval.

© Biraj